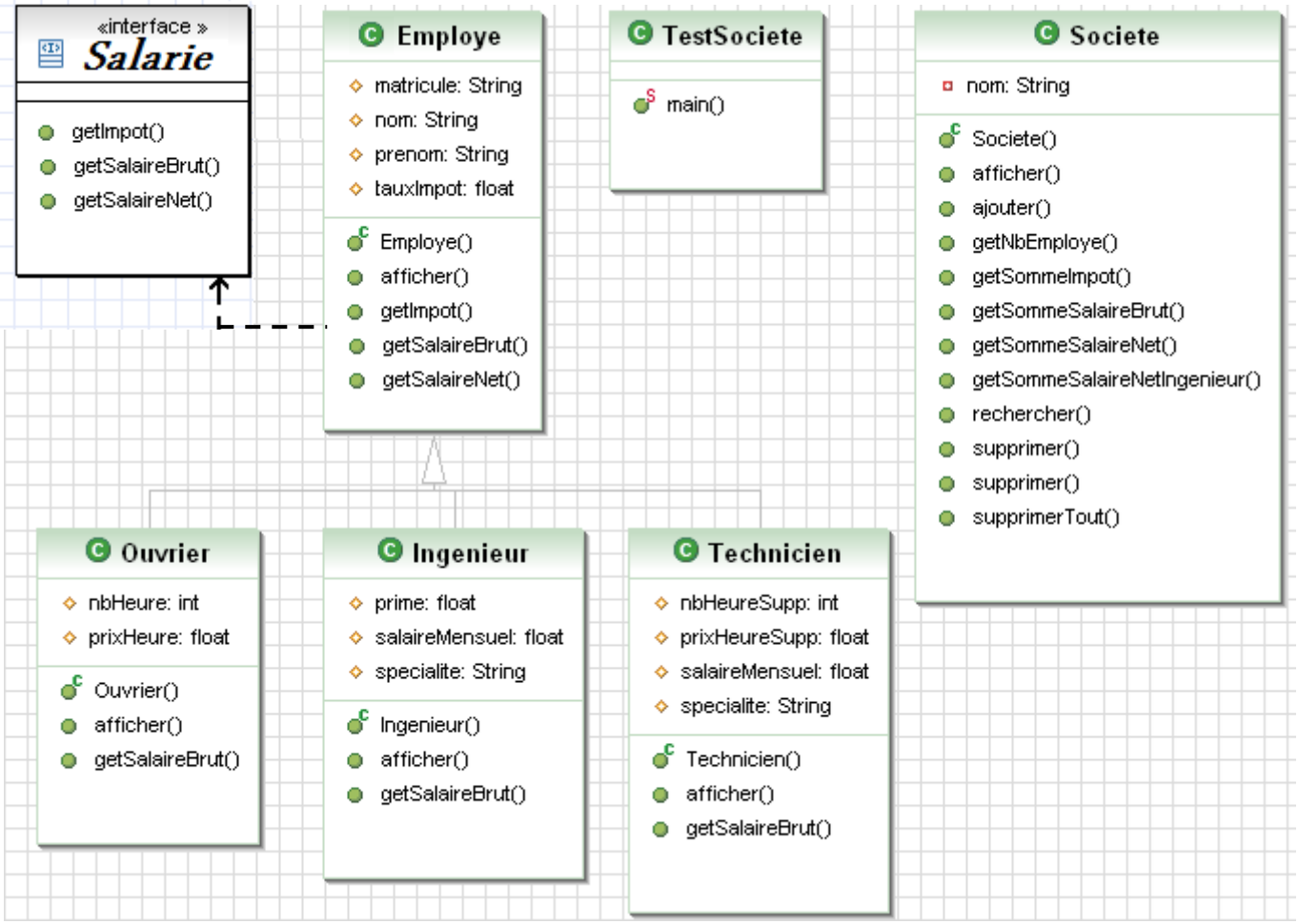
TP10

Matière : ATELIER DE PROGRAMMATION OBJET | **Classes : DSI2**

Exercice (Société)

Le diagramme de classe de la modélisation d'une société est le suivant :



Employe est une classe abstraite, elle possède une méthodes abstraite getSalaireBrut().

Les formules de calcul des salaires sont les suivantes :

salaireBrutIngenieur = salaireMensuel + prime

salaireBrutTechnicien = salaireMensuel + nbHeureSupp * prixHeureSupp

salaireBrutOuvrier = nbHeure * prixHeure

La classe Societe possède une constante MAX_EMPLOYE été en plus de l'attribut nom, elle possède deux autres attributs qui sont :

- tEmploye : tableau d'employés
- nbEmploye : nombre d'employés

1- Donner l'implémentation de l'interface et des six classes

2- Réecrire la classe Societe en utilisant un ArrayList au lieu du tableau.

La classe **ArrayList** en Java fait partie du package java.util. C'est l'une des structures de données les plus utilisées pour stocker des collections d'objets. Elle implémente l'interface List et repose sur un **tableau dynamique** redimensionnable.
Voici les points clés à retenir :

1. Caractéristiques principales

- **Taille dynamique :** Contrairement aux tableaux classiques (T[]) dont la taille est fixe à la création, un ArrayList grandit ou rétrécit automatiquement au fur et à mesure qu'on y ajoute ou retire des éléments.
- **Indexé :** Les éléments sont stockés de manière ordonnée et sont accessibles directement via leur index (commençant à 0), ce qui rend la lecture très rapide.
- **Types génériques :** Il utilise les génériques (ex: ArrayList<String>), ce qui garantit la sécurité du type à la compilation.
- **Objets uniquement :** Il ne peut stocker que des objets (références). Pour stocker des types primitifs, il faut utiliser leurs classes "wrapper" correspondantes (ex: Integer pour int, Double pour double).

2. Principales méthodes

- **add(E e)** : Ajoute un élément à la fin de la liste.
- **get(int index)** : Récupère l'élément situé à l'index spécifié.
- **set(int index, E element)** : Remplace l'élément à l'index donné par un nouveau.
- **remove(int index)** ou **remove(Object o)** : Supprime un élément.
- **size()** : Renvoie le nombre d'éléments actuellement présents.
- **contains(Object o)** : Vérifie si un élément est présent dans la liste.
- **clear()** : Vide complètement la liste.

3. Exemple d'utilisation

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        // Création d'une ArrayList de chaînes de caractères
        ArrayList<String> fruits = new ArrayList<>();

        // Ajout d'éléments
        fruits.add("Pomme");
        fruits.add("Banane");
        fruits.add("Orange");

        // Accès à un élément
        System.out.println(fruits.get(0)); // Affiche : Pomme

        // Parcours de la liste
        for (String fruit : fruits) {
            System.out.println(fruit);
        }

        // Suppression d'un élément
        fruits.remove("Banane");

        // Taille de la liste
        System.out.println("Il reste " + fruits.size() + " fruits.");
    }
}
```

4. Avantages et Inconvénients

- **Avantages :** Excellent pour la recherche et l'accès rapide aux éléments par leur index ($O(1)$) ainsi que pour l'ajout d'éléments à la fin.
- **Inconvénients :** Les insertions ou suppressions au milieu de la liste peuvent être coûteuses en performances ($O(n)$), car tous les éléments suivants doivent être décalés.